Problems with RNNs: Long-Term Dependency and Unstable Gradients

Contents

1. Overview . . . 2
2. Prerequisites . . . 3
3. The long-term dependency problem . . . 3
   - 3.1 Short-term memory . . . 3
   - 3.2 An example: predicting the next word . . . 3
   - 3.3 The problem on real reviews . . . 4
4. Why: the vanishing gradient through time . . . 5
   - 4.1 Short-term and long-term terms . . . 6
   - 4.2 With 100 time steps . . . 7
   - 4.3 One factor of the chain . . . 7
   - 4.4 What is special about an RNN . . . 8
   - 4.5 The trained RNN on real reviews . . . 10
5. Fixing the vanishing gradient . . . 11
6. Unstable training: the exploding gradient . . . 12
   - 6.1 How the gradient explodes . . . 12
   - 6.2 Fixing the exploding gradient . . . 13
7. Summary . . . 15
8. Sources . . . 15
9. Key terms . . . 16

Where this fits: Pipeline map step 8 (Model). See the Course map.

0 Foundations	1 Frame	2 Get data	3 Understand	4 Clean	5 Features	6 Reduce
7 Split	**8 Model**	9 Evaluate	10 Tune	11 Deploy	12 Test	13 Monitor

- **Builds on:** Vanishing gradient; Backpropagation through time (BPTT).
- **Leads to:** LSTM (long short-term memory).

1. Overview

Key point: A simple RNN has two problems: it cannot learn **long-term** (G-1126) dependencies, and its training can become unstable. Both come from one fact: during **backpropagation through time** (G-246; backpropagation run backwards over the time steps of a sequence, see **backpropagation through time** (G-246)) the gradient is multiplied by the same recurrent weights again and again, so it shrinks (vanishes) or grows (explodes) with distance.

RNNs suit **sequential data** (G-1774): text, time series, any data where a point depends on the points before it. Yet plain RNNs are rarely used in practice. They suffer from two problems:

1. **The long-term dependency problem**, caused by the **vanishing gradient** (G-2070; a gradient that shrinks towards 0, see multiplying small numbers).
2. **Unstable training** (G-2056), where the network fails to train, caused by the **exploding gradient** (G-731; a gradient that grows without limit, see multiplying large numbers).

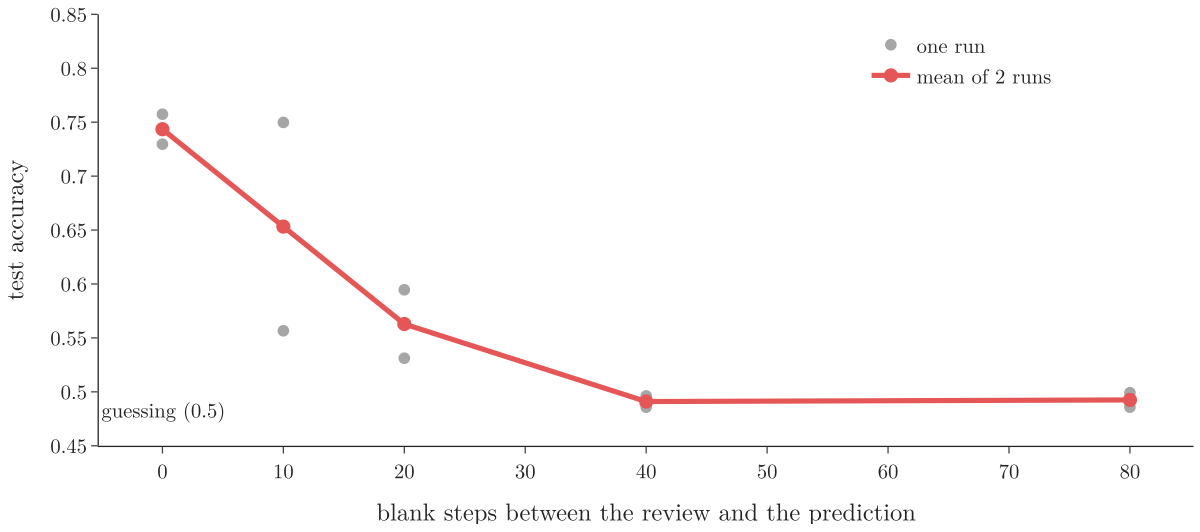


Figure 1: Test accuracy of a SimpleRNN on IMDB reviews of 50 words followed by G blank time steps before the prediction. Grey dots: single runs; red: the mean of 2 runs. The longer the gap the RNN must remember across, the closer it gets to guessing

Figure 1 shows the first problem on real data. The review is the same in every setting; only the distance between the review and the prediction grows, and the RNN loses the review's sentiment along the way. This Note explains why, and what can be done about it; the cure most used in practice, the **LSTM** (G-1123), has its own Notes.

2. Prerequisites

- The gradient for W_i in **backpropagation through time** (G-246): the gradient of W_i and W_h is a sum of one term per time step.
- Multiplying small numbers and **gradient clipping** (G-861): a long product of small factors vanishes, of large factors explodes; gradient clipping.
- The shape and slope of tanh: the slope of tanh is between 0 and 1.
- The formulas of RNN forward propagation: $h_t = \tanh(x_t W_i + h_{t-1} W_h)$.

3. The long-term dependency problem

> **Key point:** When an output depends on an input many time steps earlier, a simple RNN fails: it remembers recent inputs and forgets old ones.

3.1 Short-term memory

> **Key point:** As the number of time steps grows, the RNN cannot keep information from the early steps.

In **sequential data** (G-1774), later items depend on earlier ones. When the sequence is short, an RNN handles this well. When the sequence becomes long, with many time steps, the RNN cannot remember the early steps by the time it reaches the late ones. A simple RNN behaves like a person with short-term memory loss, such as the hero of the film *Ghajini*: recent events are clear, older ones are gone.

3.2 An example: predicting the next word

> **Key point:** "Marathi is spoken in ____" needs a word 4 steps back; the long sentence of Figure 2 needs one 22 steps back. A simple RNN handles the first and fails at the second.

Take next-word prediction, as on a phone keyboard.

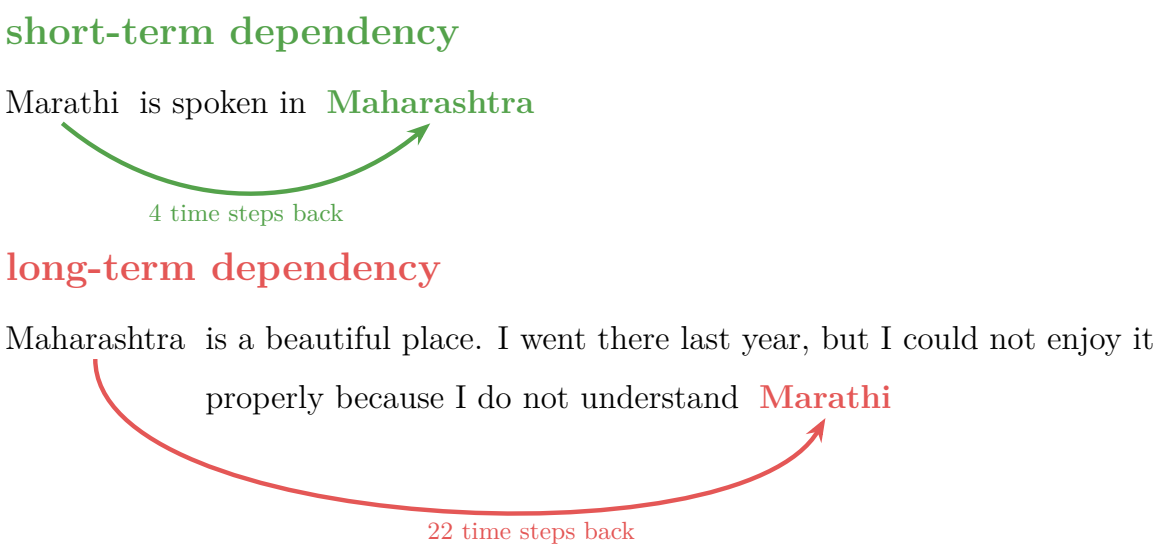


Figure 2: Two next-word predictions. The short sentence needs a word 4 time steps back; the long one needs a word 22 time steps back

- In "Marathi is spoken in Maharashtra", the word Maharashtra depends on Marathi, 4 time steps back. The dependency is **short-term**.
- In "Maharashtra is a beautiful place. I went there last year, but I could not enjoy it properly because I do not understand ____", the missing word, Marathi, depends on Maharashtra, 22 time steps back. The dependency is **long-term** (G-1126).

A keyboard built on a simple RNN performs well on sentences like the first and poorly on sentences like the second. This failure is the **long-term dependency problem**: an output at one time step depends on an input from a much earlier time step, and the RNN does not have the memory to connect them.

Bengio, Simard and Frasconi (1994) showed why gradient-based learning faces an increasingly difficult problem as the duration of the dependencies grows. Goodfellow §10.7 summarises: the probability of successfully training a traditional RNN with **stochastic gradient descent** (G-1892) rapidly reaches 0 for sequences of only length 10 or 20.

3.3 The problem on real reviews

> **Key point:** The same SimpleRNN, the same reviews; only a gap of blank steps between the review and the prediction changes. Test accuracy falls from 0.743 with no gap to 0.492 with a gap of 80 steps: guessing level.

We take IMDB movie reviews: each **observation** (G-1374; one record) is a review, and the **target** (G-1949; the output we predict) is its sentiment, positive or negative. Each review is cut to its last 50 words. Then G blank time steps (the padding token 0) are added after it, before the network gives its prediction. The network now has to carry the sentiment of the review across G steps of nothing.

Each setting is trained on 10,000 reviews for 5 epochs and tested on 5,000, twice with different seeds (Notebook). Figure 1 and the table give the mean test accuracy:

Gap G (blank steps)	0	10	20	40	80
Test accuracy	0.743	0.653	0.563	0.491	0.492

The information is in the data in every case: only the distance changes. The RNN cannot learn to carry it over a long distance. The next section shows why.

4. Why: the vanishing gradient through time

> **Key point:** The gradient of W_i is a sum of short-term terms (recent inputs) and **long-term** (G-1126) terms (distant inputs). Each long-term term contains a long product of factors $\partial h_t / \partial h_{t-1}$, which shrinks towards 0. So the weights learn almost only from recent inputs.

The idea on one weight. Take the one-node RNN of the smallest RNN, unrolled over many time steps, and look only at the feedback weight w_h. The first input travels to the last step along the feedback connection (the red arrow in the unrolled picture there), one step at a time, and every step multiplies it by w_h.

1. **In words:** one multiplication by the same weight per time step. After many steps, the first input has been multiplied by that weight many times over.
2. **Formula:** after d steps, the first input is multiplied by w_h^d.
3. **Example:** with $w_h = 2$ and 4 steps:

$$2^4 = 16$$

With 50 steps the result is a huge number:

$$2^{50} \approx 1.1 \times 10^{15}$$

With $w_h = 0.5$ and 50 steps the result is practically 0:

$$0.5^{50} \approx 8.9 \times 10^{-16}$$

The same power appears in the gradient that training sends back to the first input (sections 4.1 to 4.3 derive it). Gradient descent needs steps of a sensible size:

- A huge gradient gives huge steps. The weights jump around instead of settling at a good value: the **exploding gradient**.
- A gradient near 0 gives steps so small that the weights hardly move, and training ends before they get anywhere: the **vanishing gradient**.

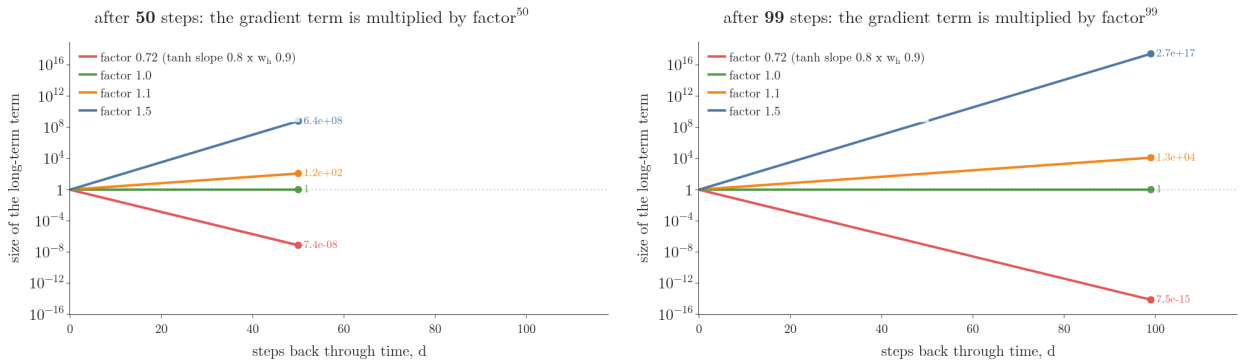


Figure 3: The same factor multiplied once per step back through time, for four factors (log scale). Below 1 the product falls towards 0 (vanishing); at 1 it stays; above 1 it grows without limit (exploding). The red line is the 0.72^{99} of section 4.3; the orange and blue lines are the two examples of section 6.1 (idea after StatQuest, "Recurrent Neural Networks (RNNs), Clearly Explained!!!")

In Figure 3 each labelled tick of the vertical (log) axis is 10,000 times the one below it (10^{-16}, 10^{-12}, 10^{-8}, ...), so a straight climb or fall on the picture means multiplying by a fixed factor at every step. Watch the lines spread apart as the steps add up: after 10 steps the largest product is about 1,500 times the smallest; after 99 steps it is more than 10^{31} times the smallest.

The rest of this section finds the exact factor for a real RNN, which is not w_h alone but w_h times the slope of tanh.

4.1 Short-term and long-term terms

> **Key point:** In the BPTT sum, the term for the last input is short; the term for the first input is the longest chain.

Take a single hidden node, so every weight is one number, and three time steps: inputs x_1, x_2, x_3, hidden states h_1, h_2, h_3, prediction $\hat{y}$. From the gradient for W_i in **backpropagation through time** (G-246), the gradient of the input weight is a sum of three terms, one for each use of w_i:

$$\frac{\partial L}{\partial w_i} = T_3 + T_2 + T_1$$

The three terms, one per time step, are:

$$T_3 = \frac{\partial L}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial h_3} \frac{\partial h_3}{\partial w_i}$$

$$T_2 = \frac{\partial L}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial h_3} \frac{\partial h_3}{\partial h_2} \frac{\partial h_2}{\partial w_i}$$

$$T_1 = \frac{\partial L}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial h_3} \frac{\partial h_3}{\partial h_2} \frac{\partial h_2}{\partial h_1} \frac{\partial h_1}{\partial w_i}$$

- The **first** term measures how the loss changes through the **last** input, x_3: a **short-term** contribution.
- The **last** term measures how the loss changes through the **first** input, x_1, the one farthest from the output: a **long-term** contribution.

Pascanu, Mikolov and Bengio (2013) call these the temporal contributions of the gradient and use the same split into **long-term** (G-1126) and short-term ones.

4.2 With 100 time steps

> **Key point:** The **long-term** (G-1126) term carries a product of 99 factors $\partial h_t / \partial h_{t-1}$.

Real sequences have 100 time steps or more. The longest term then reads

$$\frac{\partial L}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial h_{100}} \frac{\partial h_{100}}{\partial h_{99}} \frac{\partial h_{99}}{\partial h_{98}} \cdots \frac{\partial h_2}{\partial h_1} \frac{\partial h_1}{\partial w_i}$$

the second-longest stops at h_2, the third at h_3, and so on. We can write the long chain compactly with the product symbol $\prod$, which means "multiply the terms for $t = 2$ up to 100" (99 factors). With three factors:

$$\prod_{t=2}^{4} a_t = a_2 \times a_3 \times a_4$$

So the chain is:

$$\frac{\partial h_{100}}{\partial h_{99}} \cdots \frac{\partial h_2}{\partial h_1} = \prod_{t=2}^{100} \frac{\partial h_t}{\partial h_{t-1}}$$

4.3 One factor of the chain

> **Key point:** one factor of the chain is the slope of tanh times the feedback weight:
>
> $$\frac{\partial h_t}{\partial h_{t-1}} = \tanh'(\cdot)\, w_h$$

1. **In words:** h_t is tanh of something that contains $h_{t-1} w_h$. Differentiating with respect to h_{t-1} gives the slope of tanh at that point times w_h.
2. **Formula:**
$$h_t = \tanh(x_t w_i + h_{t-1} w_h)$$

Differentiate with respect to h_{t-1}:

$$\frac{\partial h_t}{\partial h_{t-1}} = \tanh'(x_t w_i + h_{t-1} w_h)\, w_h$$

so the **long-term** (G-1126) term becomes

$$\frac{\partial L}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial h_{100}} \left(\prod_{t=2}^{100} \tanh'(\cdot)\, w_h \right) \frac{\partial h_1}{\partial w_i}$$

3. **Example:** the slope of tanh is between 0 and 1 (see the slope of tanh). Suppose every slope is 0.8 and $w_h = 0.9$. Each factor is the slope times the weight, and 99 of them are multiplied.

$$0.8 \times 0.9 = 0.72$$

$$0.72^{99} \approx 7.5 \times 10^{-15}$$

   The red line of Figure 3 draws this product step by step.

The long-term term is practically 0. Its whole contribution to the gradient disappears, and the gradient is made almost entirely of short-term terms. The weights are therefore updated to fit the **recent** inputs, and the influence of distant inputs is not learned. That is the long-term dependency problem, and it is a **vanishing gradient problem** (see multiplying small numbers): the farther the input, the smaller its gradient.

> **Extra:** With several hidden nodes, each factor is a matrix, the **Jacobian** (G-980) $\partial h_t / \partial h_{t-1}$. In the row-vector form of the RNN formulas it is W_h with each column scaled by that node's tanh slope; Goodfellow (§10.2.2, eq. 10.21) writes the same thing as $W^{\mathsf{T}} \mathrm{diag}(1 - h^2)$. Pascanu et al. (2013, §2.1) prove that if the largest absolute **eigenvalue** (G-665) of the recurrent weight matrix is below $1/\gamma$, where γ bounds the slope of the activation ($\gamma = 1$ for tanh), the long-term contributions vanish as the distance grows.

4.4 What is special about an RNN

> **Key point:** A deep ANN multiplies **different** weights layer after layer; an RNN multiplies the **same** W_h at every step. A number below 1 raised to a power always vanishes, and one above 1 always explodes.

Deep ANNs already showed long products of small factors (multiplying small numbers). An RNN is worse in one specific way: the factor is the same at every step. Goodfellow §10.7 calls this problem particular to recurrent networks. Leave out tanh and the inputs, and the recurrence is $h_t = h_{t-1} W_h$, so after t steps the state, and the gradient sent back, is multiplied by W_h to the power t. The **eigenvalues** of W_h (the stretch factors of the directions that W_h only stretches and does not turn, see the vectors that do not turn) are raised to the power t: those with magnitude below 1 decay to 0, those above 1 explode (Goodfellow §10.7, eq. 10.36 to 10.39). With one node this is just w_h^t. A deep ANN with a different random

weight at each layer can be scaled to avoid the problem; the shared weight of an RNN cannot (Goodfellow §10.7).

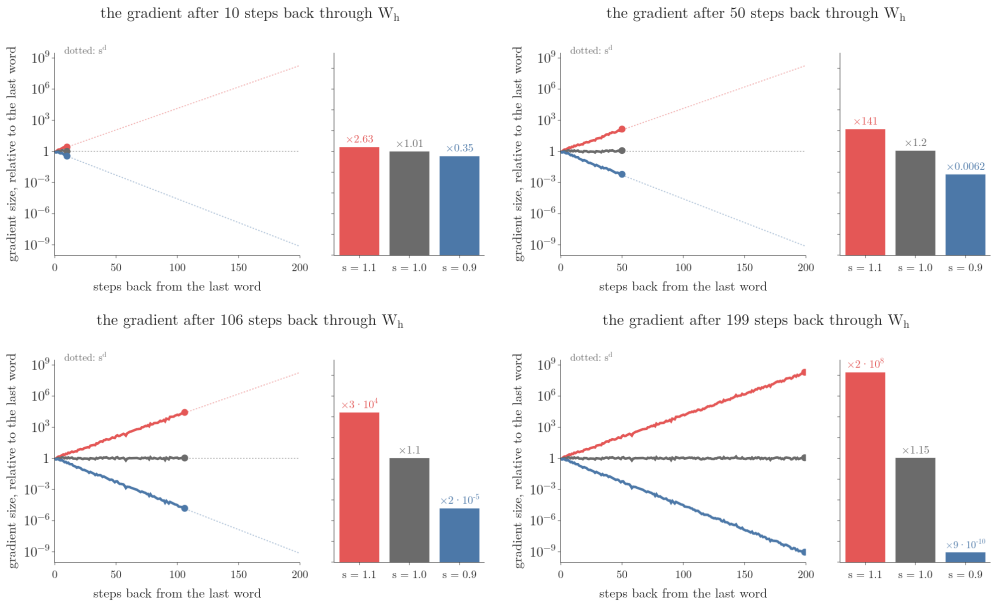


Figure 4: The gradient travelling back through an untrained SimpleRNN without activation on IMDB reviews, with W_h set to s times an orthogonal matrix (all eigenvalues of magnitude 1). Left: gradient size by distance, relative to the last word (log scale); dotted: s^d. Right: the size reached so far. Each step multiplies the gradient by about s

The Notebook tests this on real reviews. A SimpleRNN with no activation gets $W_h = s\,Q$, where Q is an **orthogonal matrix** (G-1407; a matrix that only turns vectors and never stretches them, so every **eigenvalue** (G-665) has magnitude exactly 1; see orthonormal columns), so every eigenvalue of W_h has magnitude s. Figure 4 shows the result:

s	Gradient 50 steps back, relative to the last word	s^{50}
0.9	0.0062	0.0052
1.0	1.2	1
1.1	141	117

The measured ratios follow s^{50}: only the size of the shared weight decides whether the gradient vanishes, stays or explodes. In Figure 4, watch the three solid lines stay close to their dotted s^d lines all the way to 199 steps back.

4.5 The trained RNN on real reviews

> **Key point:** In a tanh SimpleRNN trained on IMDB, a word 50 steps from the end moves the loss about a third as much as the last word, and a word 100 steps back about a seventh as much. The decay is gradual because Keras starts W_h as an orthogonal matrix.

We train a SimpleRNN with 32 nodes on 10,000 IMDB reviews (their last 200 words) for 3 epochs, with 5 different seeds; the runs reach a mean test accuracy of 0.79 (from 0.77 to 0.83). For 500 test reviews of at least 200 words, the Notebook measures, with `tf.GradientTape`, the size of the gradient of the loss with respect to each word's vector, and divides it by the size for the last word.

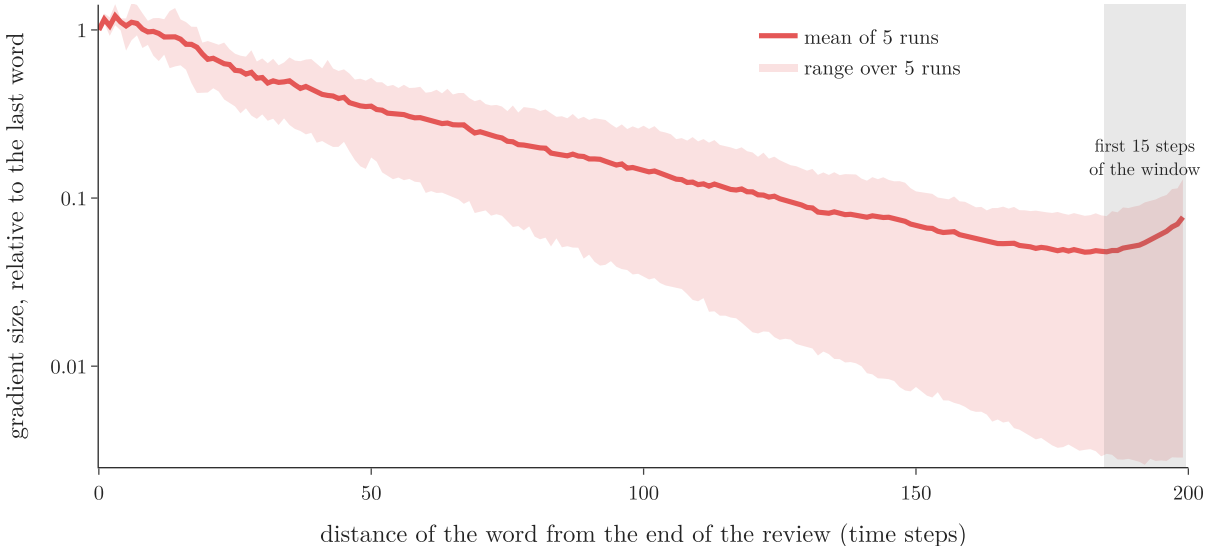


Figure 5: Gradient of the loss with respect to each word of a review, relative to the last word, for a SimpleRNN trained on IMDB (500 reviews of 200 words; red line: mean of 5 runs; band: range over the runs). Log scale. The grey strip marks the first 15 words of the 200-word window, explained at the end of this section

Distance (steps)	0	10	20	50	100	180
Relative gradient, mean of 5 runs	1	0.98	0.67	0.35	0.15	0.048

Figure 5 shows the gradient shrinking with distance: beyond the last few words, the farther a word is from the end, the weaker the training signal it sends the weights. At 180 steps, the signal is about 20 times weaker than for the last word.

The decay is slower than the 0.72^{99} of section 4.3. Keras starts W_h as an orthogonal matrix (Keras documentation, `SimpleRNN`, `recurrent_initializer="orthogonal"`), so every **eigenvalue** (G-665) of W_h has magnitude 1 at the start: the $s = 1.0$ case of Figure 4, where W_h alone neither shrinks nor grows the gradient. The shrinking left over comes from the tanh slopes, which are below 1. Starting W_h well is one of the fixes of the next section, built into Keras.

> **Extra:** *The first words of the window.* At the far right of Figure 5, the curve rises again: from 0.048 at 180 steps to 0.077 at 199. The cause is where the window starts. Each 200-word window starts from $h_0 = 0$ (the Keras default), so the first hidden states are still small: their size grows from 0.33 at the first step to 1.39 at the fifteenth, and is 1.41 to 1.71 in the rest of the window (Notebook, mean of 5 runs). A small h_t means a tanh slope $1 - h_t^2$ close to 1: 0.996 at the first step, against 0.90 to 0.94 in the rest of the window. Each backward step multiplies the gradient by W_h with its columns scaled by these slopes (section 4.3, Extra), and the gradient of a word's vector, $\partial L/\partial x_t$, is scaled by the slope at its own step too. Larger slopes shrink the gradient less, so the first words of the window get a slightly larger gradient than the words just after them.

5. Fixing the vanishing gradient

> **Key point:** Four options: a different activation, a better initialisation of W_h, skip connections through time, or an LSTM. In practice the LSTM won.

1. **A different activation function.** The slope of tanh is between 0 and 1, which shrinks every factor. **ReLU** (G-1668) and **Leaky ReLU** (G-1064; see **Leaky ReLU** (G-1064)) have slope 1 for positive inputs, so they do not shrink the factor; the factor is then w_h alone (Figure 6).
2. **Better weight initialisation.** If W_h starts with factors below 1, the product vanishes. Starting W_h as the **identity matrix** (G-915; 1 on the diagonal, 0 elsewhere), an **identity initialisation** (G-914), makes a multiplication by W_h leave the gradient unchanged at the start of training. Le, Jaitly and Hinton (2015) combined ReLU with an identity-initialised W_h and found it comparable to an LSTM on their four benchmarks.
3. **Skip connections through time** (G-1819). Connections from a state several time steps back directly to the present give the gradient shorter paths. For example, with connections that jump $d = 10$ steps, a word 100 steps back reaches the present through about 10 jumps instead of 100 single steps, so its gradient passes through about 10 shrinking factors instead of 100 (Goodfellow §10.9.1: the gradient shrinks with τ/d instead of τ, where τ is the distance in steps).
4. **Switch to an LSTM.** The LSTM was designed for exactly this problem, the "insufficient, decaying error backflow" of recurrent networks (Hochreiter and Schmidhuber 1997). Gated RNNs, the **LSTM** (G-1123) and the **GRU** (G-826), are the most effective sequence models used in practical applications (Goodfellow §10.10), so the usual choice is to move to one (see the core idea of the LSTM).

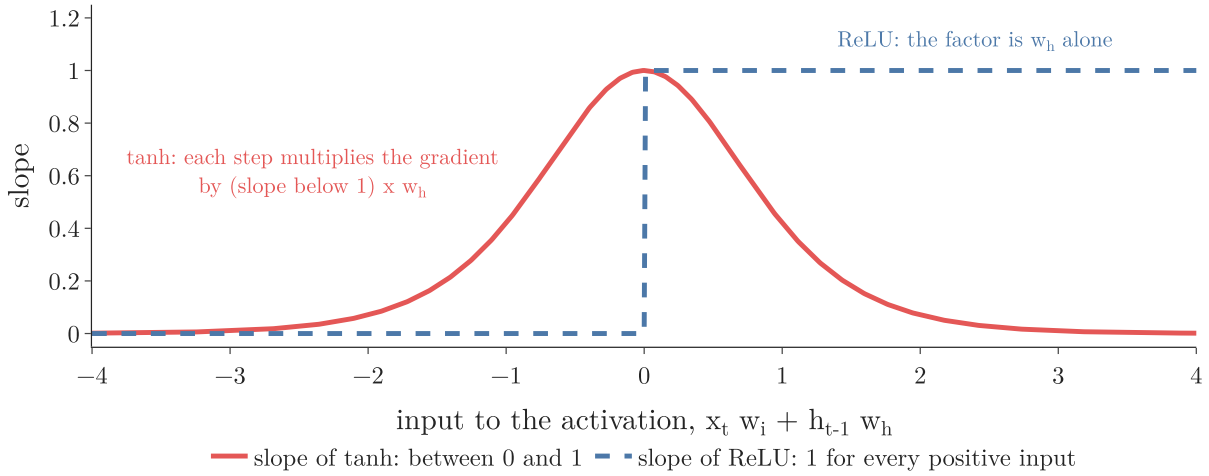


Figure 6: The slope of the activation, which multiplies w_h at every backward step. tanh's slope (red) is 1 only at 0 and smaller everywhere else, so each step shrinks the gradient. ReLU's slope (blue) is exactly 1 for every positive input, so the factor is w_h alone

In Figure 6, compare the two curves at the same input: wherever tanh's slope is below ReLU's, a tanh RNN loses more of the gradient at every step.

> **Python:** The first two fixes in Keras.
> ```
> layer = keras.layers.SimpleRNN(
>     32, activation="relu",
>     recurrent_initializer="identity")
> ```

6. Unstable training: the exploding gradient

> **Key point:** The opposite case: if the factors are above 1, the **long-term** (G-1126) terms grow huge, dominate the gradient and can reach infinity. The updates throw the weights away and the network does not train.

6.1 How the gradient explodes

> **Key point:** If each factor $\tanh'(\cdot)\, w_h$ is above 1, the product over many steps blows up.

Now the **long-term** (G-1126) terms become so large that they dominate the short-term ones, and can grow to infinity. The gradient update then becomes huge, the weights become huge or infinite, and the model does not train. The training stagnates: the loss stops improving.

1. **In words:** a factor above 1, multiplied by itself over many steps, grows without limit.
2. **Formula:** with **ReLU** (G-1668), the slope is 1 for a positive input (and 0 for a negative one), so the factor is w_h and the product over t steps is w_h^t.

3. **Example:** with $w_h = 1.1$ and 100 steps:

$$1.1^{100} \approx 13{,}781$$

With $w_h = 1.5$ and 100 steps:

$$1.5^{100} \approx 4 \times 10^{17}$$

Figure 3 (section 4) draws both examples: the orange and blue lines.

Two situations make it likely:

- **ReLU with large recurrent weights.** ReLU does not squash: its slope is 1 for every positive input. If the recurrent weights are initialised large, nothing keeps the product small, and it explodes. The $s = 1.1$ line of Figure 4 shows the same growth with no activation at all.
- **A high learning rate.** The gradient itself does not depend on the **learning rate** (G-1068), but the step does: $\Delta W = \eta\, \partial L/\partial W$. A large gradient times a large η gives a huge step, which throws the weights far away, the case shown in multiplying large numbers.

Goodfellow §10.7 notes that gradients over many steps vanish most of the time and explode rarely, but with much damage to the optimisation.

6.2 Fixing the exploding gradient

> **Key point:** Gradient clipping, a moderate learning rate, or an **LSTM** (G-1123).

1. **Gradient clipping** (G-861). Cap the size of the gradient: if its **norm** (its length as an arrow, $\sqrt{g_1^2 + g_2^2 + \ldots}$) exceeds a threshold, scale it down to the threshold, keeping its direction (Figure 7). Clipping is taught in **gradient clipping** (G-861); Pascanu et al. (2013, §3.2) proposed this norm clipping for RNNs, and Goodfellow §10.11.1 presents it as a fix for the cliffs that recurrent networks create in the loss.
2. **A controlled learning rate** (G-1068). A smaller η keeps each step small even when the gradient is large.
3. **An LSTM.** The usual choice in practice, as for the vanishing gradient. The LSTM was built to stop the error from decaying (Hochreiter and Schmidhuber 1997); gradient clipping is still the standard guard against explosions in recurrent networks (Goodfellow §10.11.1).

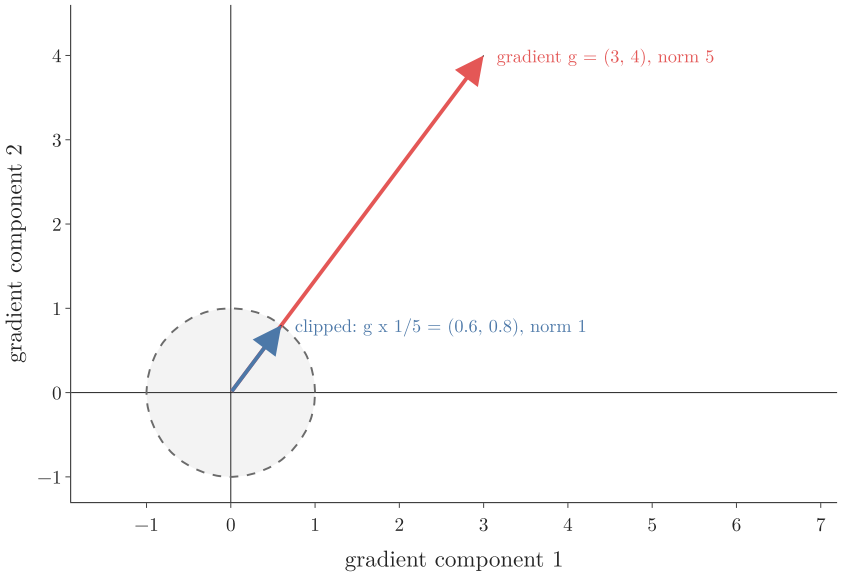


Figure 7: Clipping by norm with a threshold of 1. The gradient $g = (3, 4)$ has norm 5, so it is multiplied by $1/5$: the clipped gradient $(0.6, 0.8)$ points the same way and has norm 1

> **Python:** Clipping in Keras: every gradient is scaled down to norm at most 1 before the update.
> `opt = keras.optimizers.Adam(learning_rate=1e-3, clipnorm=1.0)`

7. Summary

	Long-term dependency	Unstable training
Cause	vanishing gradient through time	exploding gradient through time
Factor per step	activation slope $\times$ w_h below 1 (tanh: $\tanh'(\cdot)\, w_h$)	activation slope $\times$ w_h above 1 (e.g. ReLU with $w_h > 1$)
Symptom	only recent inputs influence learning	huge updates, loss stops improving or becomes NaN
Fixes	ReLU, identity initialisation of W_h, skip connections, LSTM	gradient clipping, smaller learning rate, LSTM

- Both problems come from unstable gradients: in BPTT the **long-term** (G-1126) terms contain a product of many factors $\partial h_t / \partial h_{t-1}$.
- The factor is the same W_h at every step, so the product behaves like a power: below 1 it vanishes, above 1 it explodes.
- On IMDB, a SimpleRNN's accuracy falls from 0.743 to 0.492, guessing level, when 80 blank steps separate the review from the prediction; its gradient shrinks steadily with a word's distance from the end, so the weights learn almost only from recent words.
- Simple RNNs are therefore rarely used; **LSTMs** (G-1123), which address these problems, replaced them, because the LSTM was designed to stop exactly this decay of the error signal over time.

8. Sources

Built from

- CampusX, "Problems with RNN | 100 Days of Deep Learning", YouTube, https://www.youtube.com/watch?v=AWHSZzp96kM
- StatQuest with Josh Starmer, "Recurrent Neural Networks (RNNs), Clearly Explained!!!", YouTube, https://www.youtube.com/watch?v=AsNTP8Kwu80. 11:00–15:30 (one feedback weight raised to the number of unrolled steps: 2 to the power 50 explodes, 0.5 to the power 50 vanishes; steps too large or too small).

Other references

- Bengio, Y., Simard, P. and Frasconi, P. (1994). Learning Long-Term Dependencies with Gradient Descent is Difficult. *IEEE Transactions on Neural Networks* 5(2), 157–166.
- Goodfellow, I., Bengio, Y. and Courville, A. (2016). *Deep Learning*. MIT Press. Chapter 10, §10.2.2 (gradient in an RNN), §10.7 (the challenge of long-term dependencies), §10.9.1 (skip connections through time), §10.10 (gated RNNs), §10.11.1 (clipping gradients). deeplearningbook.org/contents/rnn.html.
- Hochreiter, S. and Schmidhuber, J. (1997). Long Short-Term Memory. *Neural Computation* 9(8), 1735–1780.

- Le, Q. V., Jaitly, N. and Hinton, G. E. (2015). A Simple Way to Initialize Recurrent Networks of Rectified Linear Units. arXiv:1504.00941.
- Pascanu, R., Mikolov, T. and Bengio, Y. (2013). On the Difficulty of Training Recurrent Neural Networks. ICML 2013. arXiv:1211.5063.
- Keras API documentation: `SimpleRNN` layer (keras.io/api/layers/recurrent_layers/simple_rnn), `Identity` initializer, optimizer argument `clipnorm`.

9. Key terms

Terms taught in this Note come first; linked terms are recaps, taught in the Note the link opens.

Term	Meaning
Long-term dependency (G-1126)	An output that depends on an input many time steps earlier.
Unstable training (G-2056)	Training that does not progress because exploding gradients make the updates huge.
Identity initialisation (G-914)	Starting the recurrent weight matrix W_h as the identity matrix (1 on the diagonal, 0 elsewhere), so at the start of training multiplying by it leaves the gradient unchanged instead of shrinking it.
Skip connection through time (G-1819)	A connection from a hidden state several steps back directly to the present; it gives the gradient a shorter path, so it vanishes less.
Long-term dependency problem (G-1125)	A simple RNN's failure to link an output to an input many time steps earlier (a long-term dependency), because the gradient shrinks away as it flows back through time (the vanishing gradient through time).
Short-term and long-term contributions (G-1792)	When an RNN learns, the signal that adjusts its weights (the BPTT gradient) is a sum of two kinds of parts: short-term ones measure how the loss changes through recent inputs, long-term ones through distant inputs. The long-term ones shrink towards 0, so the RNN fails to learn from distant inputs.
Backpropagation through time (BPTT) (G-246)	Backpropagation for an RNN: the chain rule runs back over the network unfolded in time, adding up the gradient from every time step at which a shared weight is used.

Term	Meaning
Sequential data (G-1774)	Data that comes as an ordered series of pieces whose order matters, such as text or a time series; models read it one piece after another, in order.
Vanishing gradient (G-2070)	Gradients shrinking towards 0 as they pass through many layers, which slows learning.
Exploding gradient (G-731)	Gradients growing huge as they pass back through many layers, making updates erratic.
Long short-term memory (LSTM) (G-1123)	A recurrent network (RNN) that carries two memories from one time step to the next, a long-term one (the cell state) and a short-term one (the hidden state), with gates that control what each memory keeps and passes on.
Gradient clipping (G-861)	Capping the gradient before each update: if its overall size (its norm) is above a chosen limit, it is scaled down to that limit with its direction kept, so exploding gradients cannot make huge weight jumps.
Stochastic gradient descent (SGD) (G-1892)	Gradient descent that uses one random row for every update, so each update is cheap and many updates happen per epoch, at the cost of a noisier path.
Observation (G-1374)	One record of a dataset: one row of the data table.
Target (G-1949)	The output column we predict, such as the class; also called the label.
Jacobian (G-980)	The table (matrix) of all first partial derivatives of a function with several inputs and outputs, one row per output and one column per input, $J_{ij} = \partial f_i / \partial x_j$; it shows how every output changes with every input near a point.
Eigenvalue (G-665)	The number by which a matrix stretches or shrinks one of its eigenvectors: $A\mathbf{v} = \lambda\mathbf{v}$. In PCA, the eigenvalues of the covariance matrix are the variances along the principal components.
Orthogonal matrix (G-1407)	A square matrix that only rotates or flips: its columns have length 1 and are all perpendicular (orthonormal), and its inverse is its transpose.

Term	Meaning
ReLU (G-1668)	Rectified linear unit, the activation $\max(0, z)$: it passes a positive input unchanged and turns a negative one into 0; the usual default activation for hidden layers.
Leaky ReLU (G-1064)	A ReLU variant with a small slope for negative inputs, so nodes do not die.
Identity matrix (G-915)	The matrix that leaves every vector unchanged.
Gated recurrent unit (GRU) (G-826)	An RNN architecture with one memory (the hidden state) and two gates, reset and update.
Learning rate (η) (G-1068)	How strongly each update changes the model; in gradient descent, the number the slope is multiplied by to get the step size.